

BailonGO

Guía de instalación y primera ejecución

Todo lo que necesitas instalar y cada botón que debes presionar para clonar el repositorio y correr la app en Android Studio, desde cero.

Repositorio	github.com/TonyRV8/BailonGO-Flutter
Framework	Flutter 3.44.4 · Dart 3.12.2 (canal stable)
Backend	Firebase (Auth · Firestore · Storage)
Visión	MediaPipe Tasks Vision 0.10.14 (PoseLandmarker)
Plataforma	Android 7.0 (API 24) o superior

Contenido

- 1. Qué necesitas antes de empezar**
Hardware, sistema y espacio en disco
- 2. Instalar Git**
Para clonar el repositorio
- 3. Instalar el SDK de Flutter**
Descarga, descompresión y PATH
- 4. Instalar Android Studio**
IDE, SDK de Android y plugins Flutter/Dart
- 5. Configurar el SDK de Android**
Platform 36, NDK, command-line tools y licencias
- 6. Verificar con `flutter doctor`**
Todo en verde antes de continuar
- 7. Clonar el repositorio**
Y qué NO hay que configurar (Firebase ya viene listo)
- 8. Abrir el proyecto en Android Studio**
La carpeta correcta y el Gradle Sync
- 9. Preparar el dispositivo**
Celular físico (recomendado) o emulador
- 10. Correr la app**
Botón Run, Hot Reload y equivalentes por consola
- 11. Primer uso dentro de la app**
Registro, catálogo y evaluación de un paso
- 12. Solución de problemas**
Errores comunes y su fix exacto
- 13. Comandos de referencia rápida**
Chuleta para el día a día

1 Qué necesitas antes de empezar

Requisito	Detalle
Sistema operativo	Windows 10 (64 bits, versión 1809 o superior) o Windows 11. También funciona en macOS y Linux; esta guía usa las rutas y botones de Windows.
Espacio en disco	Mínimo 25 GB libres. Flutter ≈ 3 GB, Android Studio + SDK ≈ 12 GB, caché de Gradle y Pub ≈ 6 GB, el proyecto compilado ≈ 2 GB.
RAM	8 GB funciona; 16 GB recomendado. Gradle está configurado para usar hasta 8 GB (ver problema «Gradle sin memoria» si tienes 8 GB).
Internet	La primera compilación descarga ~2 GB entre Gradle, dependencias de Android y paquetes de Dart.
Dispositivo	Celular Android físico con Android 7.0 (API 24) o superior. La app usa cámara + MediaPipe en tiempo real: el emulador sirve para navegar la interfaz, pero la evaluación de pose no funciona bien con la cámara simulada.
Cuenta Firebase	No necesitas ninguna. El proyecto ya trae la configuración del backend en el repositorio.

Convención de este documento

Los bloques oscuros son comandos que escribes en una terminal. En Windows abre **PowerShell** (tecla Windows → escribe «PowerShell» → Enter). Las líneas que empiezan con **#** son comentarios, no las escribas.

2 Instalar Git

1. Descarga desde <https://git-scm.com/download/win> (se descarga solo el instalador de 64 bits).
2. Ejecuta el `.exe` y presiona **Next** en todas las pantallas. Las opciones por defecto están bien.
3. Cierra y vuelve a abrir PowerShell, y verifica:

```
git --version
# Debe imprimir algo como: git version 2.47.1.windows.1
```

Configura tu identidad (se usa en los commits):

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@example.com"
```

Usa exactamente la versión 3.44.4 (stable)

El proyecto se compiló y probó con Flutter 3.44.4 / Dart 3.12.2. Una versión distinta puede fallar en el paso de Gradle.

No corras `flutter upgrade`

sin avisar al equipo.

3.1 Descargar

1. Abre <https://docs.flutter.dev/get-started/install/windows/mobile>
2. Descarga el ZIP del SDK. Si quieres exactamente la 3.44.4, entra a <https://docs.flutter.dev/release/archive>, sección **Stable channel (Windows)**, y baja `flutter_windows_3.44.4-stable.zip`.

3.2 Descomprimir

Extrae el ZIP de modo que quede la carpeta:

```
C:\src\flutter
```

No lo pongas en estas rutas

- `C:\Program Files\...` → requiere permisos de administrador y Flutter falla.
- Rutas con espacios o acentos (`C:\Users\Mi Usuario\Descargas\...`) → rompen Gradle.
- Carpetas sincronizadas con OneDrive → corrompen la caché del SDK.

3.3 Agregar Flutter al PATH

1. Tecla Windows → escribe «**variables de entorno**» → abre `Editar las variables de entorno de esta cuenta`.
2. En el cuadro de arriba («Variables de usuario»), selecciona **Path** → `Editar...`
3. `Nuevo` → escribe: `C:\src\flutter\bin`
4. `Aceptar` en las tres ventanas.
5. **Cierra todas las terminales abiertas** y abre una nueva (el PATH sólo se refresca en procesos nuevos).

```
flutter --version
# Flutter 3.44.4 • channel stable • https://github.com/flutter/flutter.git
# Tools • Dart 3.12.2
```

Si dice «flutter no se reconoce como un comando»

El PATH no quedó. Vuelve al paso 3.3 y confirma que la ruta termina en `\bin` y que abriste una terminal *nueva*.

4

Instalar Android Studio

1. Descarga desde <https://developer.android.com/studio> y ejecuta el instalador con las opciones por defecto.
2. Al abrirlo por primera vez, el asistente **Setup Wizard** te pregunta el tipo de instalación: elige **Standard** → **Next** → acepta las licencias → **Finish** . Descargará el SDK de Android base (~5 GB, tarda).

4.1 Instalar los plugins de Flutter y Dart

1. En la pantalla de bienvenida de Android Studio: **Plugins** (menú izquierdo). Si ya tienes un proyecto abierto: **File** → **Settings** → **Plugins** .
2. Pestaña **Marketplace** → busca **Flutter** → **Install** .
3. Te preguntará si también instala **Dart** → **Yes** (es obligatorio).
4. **Restart IDE** .

Cómo saber que quedó

Al reiniciar, la pantalla de bienvenida muestra la opción **New Flutter Project** .

5

Configurar el SDK de Android

Abre el **SDK Manager**: pantalla de bienvenida → **More Actions** → **SDK Manager** . (Con un proyecto abierto: **Tools** → **SDK Manager** .)

5.1 Pestaña «SDK Platforms»

Marca la casilla **Android API 36** (el proyecto compila contra `compileSdk 36`).

5.2 Pestaña «SDK Tools»

Activa arriba a la derecha la casilla **Show Package Details** y marca:

Paquete	Versión / nota
Android SDK Build-Tools	la más reciente
Android SDK Command-line Tools (latest)	Obligatorio — sin esto <code>flutter doctor</code> falla
Android SDK Platform-Tools	incluye <code>adb</code>
NDK (Side by side)	28.2.13676358 — es la versión que exige el proyecto
CMake	la más reciente
Google USB Driver	sólo Windows, para conectar el celular por USB
Android Emulator	sólo si vas a usar emulador

Presiona `Apply` → `OK` y espera la descarga.

5.3 Aceptar las licencias del SDK

En PowerShell:

```
flutter doctor --android-licenses
# Responde "y" y Enter a cada licencia hasta que termine.
```

6 Verificar con `flutter doctor`

```
flutter doctor -v
```

Necesitas ver una palomita (`[v]`) en estas tres líneas:

```
[v] Flutter (Channel stable, 3.44.4, on Microsoft Windows ...)
[v] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[v] Android Studio (version 2025.x)
```

Se pueden ignorar sin problema

- `[X]` Xcode y `[X]` CocoaPods → sólo aplican a iOS/macOS.
- `[X]` Visual Studio → sólo aplica si compilas la app para Windows escritorio.
- `[!]` Chrome → sólo aplica para Flutter Web.

Nosotros corremos en

Android

: con esas tres palomitas es suficiente.

6.1 Java (JDK)

El proyecto usa **Java 17** como nivel de compilación y Android Gradle Plugin 9, que exige **JDK 17 o superior**.

Android Studio ya trae su propio JDK (JetBrains Runtime 21), y Flutter lo detecta solo. Si `flutter doctor -v` no lo encuentra, apúntalo a mano:

```
# Ajusta la ruta a donde instalaste Android Studio
flutter config --jdk-dir "C:\Program Files\Android\Android Studio\jbr"
```

7

Clonar el repositorio

Elige una carpeta sin espacios ni acentos en la ruta. Ejemplo con `C:\dev` :

```
cd C:\
mkdir dev
cd C:\dev
git clone https://github.com/TonyRV8/BailonGO-Flutter.git
cd BailonGO-Flutter
```

Descarga las dependencias de Dart declaradas en `pubspec.yaml` :

```
flutter pub get
```

Lo que NO tienes que hacer (ya viene en el repositorio)

- **No**

crees un proyecto de Firebase, ni corras `flutterfire configure` . Las credenciales de cliente ya están en `android/app/google-services.json` y `lib/core/config/firebase_options.dart` : todos apuntamos al mismo backend `bailongo-f3384` .

- **No**

corras `flutter create .` . Las carpetas `android/` , `ios/` y `web/` ya están generadas y modificadas a mano (permisos de cámara, MediaPipe, minSdk 24). Volver a generarlas

borra esos cambios

- **No**

descargues el modelo de pose. `android/app/src/main/assets/pose_landmarker_lite.task` (5.7 MB) ya está versionado.

- **No**

busques los videos de referencia por fuera. `assets/videos/ref.mov` y `assets/fixtures/{ref,good,bad}.mov` están en el repositorio y la app los empaqueta al compilar.

- **No**

edites `android/local.properties` : lo genera Flutter con la ruta de *tu* máquina y por eso está en `.gitignore` .

El repositorio pesa ~40 MB

Los cuatro videos `.mov` y el modelo `.task` son la mayor parte. Es normal que el `git clone` tarde un par de minutos.

8

Abrir el proyecto en Android Studio

1. Abre Android Studio → **Open** (pantalla de bienvenida) o **File** → **Open...**
2. Selecciona la carpeta **raíz** del proyecto:

C:\dev\BailonGO-Flutter

Error clásico: abrir la carpeta equivocada

Selecciona **BailonGO-Flutter**,
no

BailonGO-Flutter\android. Si abres **android/**, Android Studio lo trata como proyecto Android puro, no verás el código Dart y el botón Run no compilará la app de Flutter.

3. Si pregunta **Trust Project?** → **Trust Project**.
4. Espera a que termine la barra de progreso de abajo. En la primera apertura corre dos procesos largos:
 - **Pub get** — descarga los paquetes de Dart (1–2 min).
 - **Gradle sync** — descarga Gradle 9.1.0, el Android Gradle Plugin 9.0.1, Kotlin 2.3.20, MediaPipe y las librerías de Firebase. **Esto tarda de 10 a 25 minutos la primera vez.**
5. Cuando la barra inferior quede vacía y no haya errores en rojo en la pestaña **Build**, el proyecto está listo.

Si el Gradle sync no arranca solo

Menú **File** → **Sync Project with Gradle Files** (o el ícono del elefante con la flecha azul, arriba a la derecha).

9

Preparar el dispositivo

9.1 Opción A — Celular Android físico (recomendado)

1. En el celular: **Ajustes** → **Acerca del teléfono** → toca **7 veces** sobre **Número de compilación**. Aparece «Ya eres desarrollador».
2. Regresa a **Ajustes** → **Sistema** → **Opciones de desarrollador** → activa **Depuración por USB**.
3. Conecta el celular por cable USB a la computadora.
4. En el celular saldrá el diálogo «¿Permitir depuración USB?» → marca **Permitir siempre desde esta computadora** → **Permitir**.
5. Verifica desde PowerShell:

```
flutter devices
# Debe listar tu celular, por ejemplo:
# SM A536B (mobile) • RF8T20XXXXX • android-arm64 • Android 14 (API 34)
```

Si no aparece el celular

Cambia el modo de conexión USB del celular a

Transferencia de archivos (MTP)

(baja la barra de notificaciones y toca «Cargando este dispositivo por USB»). Prueba otro cable — muchos cables baratos son sólo de carga y no transmiten datos.

9.2 Opción B — Emulador

Sirve para revisar el catálogo, login y navegación, pero la evaluación con cámara no dará resultados reales.

1. **Tools** → **Device Manager** → **+** → **Create Virtual Device** .
2. Elige **Pixel 6** → **Next** .
3. Descarga una imagen de sistema con **API 34 o superior** (nunca menor a 24) → **Next** → **Finish** .
4. Presiona el botón ► del emulador en el Device Manager para arrancarlo.

10.1 Desde Android Studio

1. En la barra superior, abre el **selector de dispositivo** (el desplegable a la izquierda del botón verde ▶) y elige tu celular o emulador.
2. A su izquierda, en el selector de configuración, debe decir **main.dart**.
3. Presiona el botón verde  **Run** — o el atajo **Shift + F10**.
4. La primera compilación tarda **5 a 15 minutos** (Gradle compila MediaPipe y Firebase desde cero). Las siguientes bajan a menos de un minuto.
5. Cuando la app abra en el dispositivo, el panel  mostrará la línea de DevTools y la app quedará en modo **debug**.

Acción	Atajo	Para qué sirve
Run	Shift + F10	Compila e instala la app.
Hot Reload ⚡	Ctrl + \	Aplica cambios de UI en menos de un segundo, sin perder el estado ni reiniciar. Úsalo el 90 % del tiempo.
Hot Restart	Ctrl + Shift + \	Reinicia la app conservando la compilación. Necesario si tocas <code>main.dart</code> , providers de Riverpod o código nativo.
Stop ■	Ctrl + F2	Detiene la sesión de debug.

10.2 Desde la terminal (equivalente)

```
# Desde la raíz del proyecto
flutter run

# Si tienes varios dispositivos conectados, elige uno:
flutter devices
flutter run -d RF8T20XXXXX

# Con la sesión corriendo, escribe en la misma terminal:
#   r → hot reload      R → hot restart
#   q → salir           v → abrir DevTools
```

10.3 Correr las pruebas unitarias

```
flutter test
# Prueban el motor de evaluación: DTW, extracción de features y feedback.
```

1. **Registro.** La app abre en el login. Toca «**Crear cuenta**» y regístrate con cualquier correo y una contraseña de 6+ caracteres. Es una cuenta real de Firebase Authentication; cada quien usa la suya y la sesión queda guardada entre reinicios.
2. **Catálogo.** Al entrar verás los pasos de cumbia nivel Bronce más un **paso de prueba**. Si Firestore aún no tiene el catálogo, la app usa la copia local (`catalog_seed.dart`) — es el comportamiento esperado y no es un error.
3. **Detalle del paso.** Toca un paso para ver su descripción y el video de referencia empaquetado en la app.
4. **Evaluación.** Desde el detalle entra a evaluar. Android pedirá permiso de **Cámara** → **Permitir** . Colócate a cuerpo completo dentro del encuadre, con buena luz, y baila el paso.
5. **Resultados.** Al terminar aparece la hoja de resultados con la calificación y la realimentación por segmento.

11.1 Herramientas de desarrollo (menú Configuración)

En el Drawer → **Configuración** hay utilidades marcadas (dev) :

Opción	Qué hace
Umbral de «Aprendido»	Ajusta la calificación mínima para dar un paso por aprendido.
Corregir lateralidad (espejo)	Invierte izquierda/derecha si la cámara frontal espeja la imagen.
Sembrar catálogo (dev)	Escribe el catálogo local en Firestore.
Subir referencia (dev)	Extrae las features del paso de prueba y las guarda en Firestore.
Validar motor (dev)	Corre <code>good.mov</code> y <code>bad.mov</code> contra <code>ref.mov</code> y muestra los scores. Es la forma más rápida de comprobar que MediaPipe y el motor DTW funcionan en tu dispositivo , sin necesidad de bailar.
Borrar progreso de prueba (dev)	Elimina tus intentos guardados.

Error: `flutter.sdk not set in local.properties`

Gradle no sabe dónde está el SDK de Flutter. Ocurre si compilas desde `android/` antes de que Flutter genere el archivo.

```
# Desde la RAÍZ del proyecto (no desde android/)
flutter pub get
flutter run
```

Si persiste, crea a mano `android/local.properties` con tu ruta (nota las barras dobles):

```
flutter.sdk=C:\\src\\flutter
sdk.dir=C:\\Users\\TU_USUARIO\\AppData\\Local\\Android\\Sdk
```

Error: `File google-services.json is missing`

El archivo debe existir en `android/app/google-services.json`. Ya viene en el repositorio, así que si falta es porque tienes un clon viejo:

```
git pull
```

Error: `Unsupported class file major version` o **problemas de JDK**

Estás compilando con un Java menor a 17. Apunta Flutter al JDK de Android Studio:

```
flutter config --jdk-dir "C:\\Program Files\\Android\\Android Studio\\jbr"
flutter clean
flutter run
```

Error: `NDK ... did not have a source.properties file`

Falta la versión exacta del NDK. SDK Manager → `SDK Tools` → `Show Package Details` → marca **NDK 28.2.13676358** → `Apply`.

Gradle se queda sin memoria / la compilación congela la PC

`android/gradle.properties` pide hasta 8 GB de heap. Si tu máquina tiene 8 GB de RAM, bájalo **en tu copia local** a:

```
org.gradle.jvmargs=-Xmx4G -XX:MaxMetaspaceSize=1G
```

No subas ese cambio al repositorio.

Es un ajuste de tu máquina; si lo commiteas, se lo impones a todo el equipo.

Error de `minSdk` / la app no instala en el celular

MediaPipe exige **Android 7.0 (API 24)** como mínimo. Un dispositivo o emulador con API menor no puede instalar la app. Verifica con `flutter devices`.

La cámara se ve negra o no detecta el cuerpo

- Confirma que diste el permiso: Ajustes de Android → Apps → **bailongo** → Permisos → Cámara → **Permitir**.
- Necesita **cuerpo completo dentro del encuadre** y buena iluminación; MediaPipe pierde los landmarks con poca luz o si te sales del cuadro.
- En emulador la cámara simulada no produce una pose válida. Usa celular físico.

Errores raros después de un `git pull`

Limpia todo y vuelve a compilar. Resuelve la mayoría de los problemas de caché:

```
flutter clean
flutter pub get
flutter run
```

Si aun así falla, limpia también la caché de Gradle del proyecto:

```
cd android
.\gradlew clean
cd ..
flutter run
```

Firebase truena con `UnsupportedError` al iniciar

Significa que `lib/core/config/firebase_options.dart` quedó como plantilla vacía. No debería pasar (está versionado con los valores reales). Si pasa, **no corras** `flutterfire configure`: avisa al equipo, porque generaría credenciales de otro proyecto y dejaría de compartirse la base de datos.

Git avisa de saltos de línea (`LF will be replaced by CRLF`)

Es una advertencia, no un error. El repositorio normaliza a LF vía `.gitattributes`. Ignórala.

La primera compilación va lentísima

Es normal: 10–25 min de Gradle sync + 5–15 min de build. Ayuda mucho excluir la carpeta del proyecto, `C:\src\flutter` y `~\.gradle` del análisis en tiempo real del antivirus.

```
- Diagnóstico -
flutter doctor -v          # estado del entorno
flutter devices            # dispositivos conectados
flutter --version          # versión del SDK

- Día a día -
flutter pub get             # instalar dependencias de pubspec.yaml
flutter run                # compilar y correr en debug
flutter run -d <device-id> # elegir dispositivo
flutter test               # pruebas unitarias
flutter analyze            # linter estático

- Cuando algo se rompe -
flutter clean              # borrar artefactos de build
flutter pub cache repair   # reparar caché de paquetes

- Git -
git pull                  # traer los últimos cambios
git status                # ver qué modificaste
git checkout -b mi-rama   # crear rama de trabajo

- Generar el APK de release (para compartir la app) -
flutter build apk --release
# queda en build\app\outputs\flutter-apk\app-release.apk
```

Estructura del proyecto

```
BailonGO-Flutter/
├─ lib/
│  ├─ main.dart           # arranque: Firebase + persistencia offline
│  ├─ app.dart            # MaterialApp.router
│  ├─ core/               # tema, router, config, utils
│  └─ features/
│     ├─ auth/            # login, registro, sesión
│     ├─ catalog/         # pasos de baile + videos
│     ├─ pose/            # cámara + MediaPipe PoseLandmarker
│     ├─ evaluation/      # motor DTW, features, resultados
│     └─ profile/ settings/ # perfil y configuración
├─ android/app/src/main/
│  ├─ assets/pose_landmarker_lite.task # modelo MediaPipe
│  ├─ kotlin/.../MainActivity.kt      # platform channel de pose
│  └─ google-services.json            # config Firebase (versionada)
├─ assets/
│  ├─ videos/ref.mov                # video de referencia del catálogo
│  └─ fixtures/{ref,good,bad}.mov   # fixtures para validar el motor
├─ test/evaluation/                # pruebas unitarias del motor
├─ firestore.rules storage.rules   # reglas de seguridad del backend
└─ pubspec.yaml                   # dependencias y assets
```

Checklist final — lo lograste si...

1. `flutter doctor -v` muestra palomita en Flutter, Android toolchain y Android Studio.
2. `flutter devices` lista tu celular.
3. El botón ► de Android Studio instala la app y abre la pantalla de login.
4. Puedes registrarte y ver el catálogo de pasos.
5. Configuración →
Validar motor (dev)
devuelve scores sin error.